

The Productivity Paradox of Generative AI in Software Development

- [The Productivity Paradox of Generative AI in Software Development](#)
 - [A Complexity Displacement Analysis Grounded in Startup Practice](#)
 - [Abstract](#)
 - [List of Tables and Figures](#)
 - [1. Introduction: The Solow Paradox and Large Language Models](#)
 - [1.1 The Original Paradox](#)
 - [1.2 A New Paradox Emerges](#)
 - [1.3 Research Question](#)
 - [1.4 Thesis Statement](#)
 - [2. Theoretical Foundations](#)
 - [2.1 The Task-Based View of AI and Labor](#)
 - [2.2 The Productivity J-Curve](#)
 - [2.3 From Productivity to Complexity](#)
 - [3. Research Methodology](#)
 - [3.1 Research Design](#)
 - [3.2 Literature Synthesis](#)
 - [3.3 Reflexive Case Study: FytIY](#)
 - [3.4 Cross-Case Validation: TaskOrbit](#)
 - [4. New Bottlenecks and Operational Constraints](#)
 - [4.1 The Cognitive Verification Tax](#)
 - [4.2 Session Drift](#)
 - [4.3 Budgeted Intelligence: The Economics of AI-Assisted Development](#)
 - [5. Synthesis: Modeling Complexity Displacement in React Native Development](#)
 - [5.1 Quantifying Displacement: A Task-Level Analysis](#)
 - [5.2 The Complexity Displacement Model](#)
 - [5.3 The "Comprehension Debt" Dimension](#)
 - [5.4 Implications for the Startup Context](#)
 - [6. Discussion](#)
 - [6.1 Reframing the Debate](#)
 - [6.2 Toward a "Complexity-Aware" Framework](#)
 - [6.3 Limitations](#)
 - [7. Conclusion](#)
 - [References](#)
 - [Appendix A: FytIY Codebase Summary](#)
 - [Appendix B: AI Tools Used During Development](#)
 - [Appendix B.1: Cursor Skills Configuration](#)
 - [Appendix C: Task-Level Productivity Estimates](#)

- [Appendix D: Git-Derived Development Statistics](#)
 - [Monthly Commit Velocity](#)
- [Author Note & Replication](#)

The Productivity Paradox of Generative AI in Software Development

A Complexity Displacement Analysis Grounded in Startup Practice

Author: Shikhar Thakur

Affiliation: Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU)

Program: M.Sc. International Information Systems

Contact: shikharthakur2404@gmail.com

Status: Independent Research Preprint / Working Paper

Date: May 2026

Abstract

Generative AI (GenAI) tools promise transformative productivity gains in software engineering. Controlled experiments report 26–55% faster task completion, yet a growing body of evidence suggests that experienced developers may be slower with AI assistance, and that organizational delivery metrics show no clear improvement. This paper investigates this contradiction through the lens of the Solow Productivity Paradox, arguing that GenAI does not eliminate complexity but displaces it — from execution to verification, orchestration, and resource management. Drawing on recent IS research, productivity studies, and a reflexive case analysis of a React Native startup (99,000 lines of code, 93 API integrations, 2,111 commits over 209 active coding days), the paper introduces the *Complexity Displacement Thesis*: GenAI produces significant local efficiency gains that are systematically offset by a Cognitive Verification Tax, Session Drift, and the economics of Budgeted Intelligence. The paper concludes that the productivity paradox persists not because the technology fails, but because sociotechnical systems have not yet evolved the review norms, architectural boundaries, and prompt discipline required to absorb the complexity that GenAI introduces.

Keywords: Productivity Paradox, Generative AI, Software Engineering, Complexity Displacement, LLM, React Native, Solow Paradox

List of Tables and Figures

- **Table 1:** Developer Belief vs. Measured Reality (Becker et al., 2025)
 - **Table 2:** FytlY Tool-to-Model Allocation Strategy
 - **Table 3:** FytlY Codebase Evolution Phases
 - **Table 4:** FytlY Task-Level Productivity and Verification Estimates
 - **Table 5:** Cross-Case Structural Comparison (FytlY vs. TaskOrbit)
 - **Table 6:** TaskOrbit Quantified Complexity Displacement Patterns
 - **Table A.1:** FytlY Codebase Metrics Summary
 - **Table B.1:** Taxonomy of GenAI Tools Utilized
 - **Table C.1:** Granular Task-Level Effort and Net Productivity Gains
 - **Table D.1:** FytlY Git History Quantitative Statistics
 - **Table D.2:** FytlY Monthly Commit Velocity and Development Focus
-

1. Introduction: The Solow Paradox and Large Language Models

1.1 The Original Paradox

In 1987, the Nobel laureate Robert Solow observed that “you can see the computer age everywhere but in the productivity statistics” (Solow, 1987). This remark crystallized what became known as the *Productivity Paradox of Information Technology* — the persistent observation that massive investments in IT infrastructure fail to produce commensurate gains in measurable productivity. Brynjolfsson (1993) formalized this observation, demonstrating that despite over \$1 trillion in U.S. IT spending during the 1980s, aggregate productivity growth remained flat or declining.

The paradox was widely considered resolved by the late 1990s, as the delayed diffusion of IT — combined with complementary organizational innovations — finally produced measurable productivity acceleration (Brynjolfsson & Hitt, 2000). However, the underlying mechanism was never a simple story of “invest, then wait.” Rather, value materialized only when organizations co-evolved their processes, skills, and management structures to absorb the capabilities that IT introduced (David, 1990).

1.2 A New Paradox Emerges

Nearly four decades later, we face a highly analogous situation. Generative AI — particularly Large Language Models (LLMs) such as GPT-4, Claude, and Gemini — has been positioned as a potential exception to the paradox. Unlike prior IT investments that primarily automated mechanical or routine cognitive tasks, GenAI can generate novel text, code, and analysis, seemingly automating the creative-cognitive labor that constitutes the core of knowledge work (Eloundou et al., 2023).

In software engineering specifically, industry adoption has been rapid. GitHub reported that Copilot users completed tasks 55% faster in controlled experiments (Peng et al., 2023). Microsoft’s internal field study found a 26% increase in pull requests per week among Copilot users (Kalliamvakou, 2022). McKinsey estimated that GenAI could automate 60–70% of developer activities (Chui et al., 2023).

Yet the picture is more nuanced than these headline figures suggest. The 2025 METR randomized controlled trial — arguably the most rigorous study to date — found that experienced open-source developers were **19% slower** when using AI coding assistants on real-world tasks in their own repositories (METR, 2025). Notably, these developers *perceived* themselves to be 20% faster, despite objective evidence to the contrary. The 2026 DORA report observed that high AI adoption was associated with a slight *decrease* in delivery throughput and stability across large engineering organizations (DORA, 2026).

This paper argues that we are witnessing a new instance of the Solow Paradox — one specific to cognitive automation — and that its persistence can be explained by a mechanism I term *Complexity Displacement*.

1.3 Research Question

To what extent does Generative AI shift the bottleneck of software development from execution to verification and orchestration, and how does this “complexity displacement” explain the persistence of the productivity paradox in small-scale startups?

1.4 Thesis Statement

I argue that GenAI produces significant *local* efficiency gains — faster unit task completion, accelerated boilerplate generation, and reduced time to first draft — but *reallocates* human labor toward higher-order verification, exception handling, and toolchain management. This reallocation is not a net-zero transfer; it introduces qualitatively new forms of cognitive overhead that are poorly captured by existing productivity metrics. In a startup context, where resources are lean and feedback loops are tight, the “economics of intelligence” (rate limits, token quotas, model latency) transform AI from a seamless assistant into a *budgeted resource* that requires strategic scheduling, further complicating the productivity equation.

2. Theoretical Foundations

2.1 The Task-Based View of AI and Labor

Brynjolfsson, Mitchell, and Rock (2018) introduced the *task-based framework* for analyzing AI’s impact on labor. Rather than asking whether AI will replace *jobs*, this framework decomposes occupations into constituent tasks and evaluates the “suitability for machine learning” (SML) of each task independently. A software developer’s job, for instance, comprises tasks ranging from requirements analysis and architecture design (low SML) to writing boilerplate code and generating unit tests (high SML).

This decomposition is essential for understanding the productivity paradox because it reveals a critical asymmetry: **AI excels at the tasks that were already among the fastest for skilled humans, while remaining inadequate at the tasks that constitute genuine bottlenecks.** Writing a Redux reducer is fast with or without AI; debugging a race condition in a WebSocket handler requires deep contextual reasoning that current LLMs struggle to provide.

Acemoglu and Restrepo (2019) extend this framework by distinguishing between *automation* (replacing human labor on existing tasks) and *augmentation* (creating new tasks that require human judgment). GenAI, I argue, occupies a complex intermediary position: it automates the lower-friction parts of execution while *creating* new tasks in verification and orchestration that did not previously exist.

2.2 The Productivity J-Curve

Brynjolfsson, Rock, and Syverson (2021) proposed the *Productivity J-Curve* to explain why general-purpose technologies (GPTs) often produce an initial *decline* in measured productivity before delivering substantial gains. The mechanism is straightforward: firms must invest in complementary “intangible assets” — organizational redesign, process re-engineering, and worker retraining — before the technology’s potential can be realized. During this investment phase, output stagnates or declines even as spending increases, producing the paradox.

Applied to GenAI in software development, the J-Curve predicts that we are currently in the *trough* of the curve. Organizations are investing heavily in AI tooling (Copilot licenses, API costs, agentic platforms) but have not yet developed the complementary assets — review norms, prompt engineering expertise, architectural boundaries — that would allow these tools to deliver sustained productivity gains.

2.3 From Productivity to Complexity

The theoretical contribution of this paper lies in extending the J-Curve model with a mechanism-level explanation: **complexity displacement**. Where the J-Curve describes *when* productivity gains will materialize (after complementary investment), complexity displacement describes *where* the work goes in the interim. Specifically, I identify three mechanisms through which GenAI displaces rather than eliminates complexity:

1. **The Cognitive Verification Tax** — the mandatory cognitive labor of reviewing AI-generated output for hallucinations, subtle logic errors, and architectural mismatches.
2. **Session Drift** — the degradation of context, intent, and coherence in long-running agentic conversations, requiring human intervention to realign the AI’s trajectory.
3. **Budgeted Intelligence** — the operational constraints (token quotas, rate limits, model latency) that transform AI from an unlimited assistant into a scarce resource requiring allocation decisions.

3. Research Methodology

3.1 Research Design

This study employs a mixed-methodological research design combining **literature synthesis** of theoretical frameworks in Information Systems (IS) and software engineering with a **reflexive case study** and a **cross-case git history analysis**. The integration of these methods enables a multi-dimensional examination of the Productivity Paradox and the Complexity Displacement Thesis, connecting macro-level economic theory to micro-level engineering practices.

3.2 Literature Synthesis

The theoretical framework is constructed by synthesizing classical IS literature on the Solow Productivity Paradox (Solow, 1987; Brynjolfsson, 1993), the task-based view of labor (Brynjolfsson

et al., 2018), and the Productivity J-Curve (Brynjolfsson et al., 2021) with software engineering research on technical debt (Cunningham, 1992), software evolution (Lehman, 1980), and early empirical studies on generative AI coding tools (METR, 2025; DORA, 2026). This synthesis establishes the theoretical foundation for the Complexity Displacement Thesis, identifying the shift in bottlenecks from execution to verification and orchestration.

3.3 Reflexive Case Study: FytIY

The primary empirical base of this study is a reflexive case analysis of FytIY, an early-stage React Native health and fitness startup. Reflexive case study design is particularly suited for in-depth, practitioner-led investigations where the researcher acts as both the developer and the analyst, allowing for direct observation of cognitive states, decision-making processes, and tool-management overhead.

Data Corpus and Context: The FytIY application consists of approximately 99,000 lines of code across 621 source files, featuring 93 Redux-Saga asynchronous operations and 18 API integrations. The development activity spans 209 active coding days over a 10.5-month calendar period (July 2025 to May 2026), capturing 2,111 commits. The developer utilized a multi-model toolchain including Cursor (Tab completion and Composer modes), OpenCode (CLI terminal agent), and the Gemini CLI (see Appendix B for details).

Data Collection and Categorization: 1. *Git Commit History:* All 2,111 commits were exported and analyzed. Commits were categorized using a rule-based keyword taxonomy to distinguish between new features, bug fixes, formatting/lint adjustments, merge conflict resolutions, and state restorations (reverts). 2. *Reflective Developer Logs:* A structured development journal was maintained to document reasoning processes, tool allocation choices, rate limit occurrences, and specific technical friction points (such as sandbox testing anomalies and App Store reviews). 3. *Code Churn Analysis:* Code additions and deletions were audited monthly to measure the ratio of stable code to discarded code (churn).

3.4 Cross-Case Validation: TaskOrbit

To mitigate the generalizability limitations inherent in a single-developer reflexive case study, a cross-case validation was performed on the TaskOrbit project. TaskOrbit is a conversational agent orchestration platform built as part of a university team project (AMOS 2026, FAU).

Data Corpus and Context: TaskOrbit comprises approximately 46,000 source lines across a Python/FastAPI backend and a React/TypeScript frontend. It was developed by a distributed team of 12 contributors over a 68-day sprint, accumulating 645 commits. The team used Cursor's agent mode as their primary development tool.

Comparative Analysis: The TaskOrbit git history was systematically audited using the same commit categorization taxonomy applied to FytIY. This allowed for a direct quantitative comparison of "janitorial commits" (lint/CI/formatting), "fix-after-fix chains," "state restorations," and "cross-path desync" occurrences. The comparison validates whether the complexity displacement mechanisms

observed in the solo startup context (such as the Cognitive Verification Tax and Session Drift) generalize to a distributed team environment.

4. New Bottlenecks and Operational Constraints

4.1 The Cognitive Verification Tax

4.1.1 The Nature of the Tax

When a developer writes code manually, verification is embedded in the act of creation. Each line is typed with intent; the developer understands *why* the code exists because they authored it. GenAI inverts this relationship. The developer receives a candidate solution and must verify it without having participated in its reasoning process. This verification is not trivial — it requires understanding what the code does, what it *should* do, and what it might do incorrectly in edge cases.

A growing body of industry data quantifies this burden with increasing precision. A 2026 survey of 10,000 developers found that engineers now spend **11.4 hours per week reviewing AI-generated code** compared to 9.8 hours writing code manually — meaning verification has overtaken generation as the dominant time expenditure (Digital Applied / LinearB, 2026). Faros AI's (2026) engineering telemetry across thousands of teams confirmed this shift quantitatively: AI adoption produced a **98% increase in PRs merged** and **154% larger average PR size**, but this volume triggered a **91% increase in code review time** — while organizational delivery metrics (lead time, deployment frequency) remained flat. Opsera's 2026 benchmark across 250,000 developers confirmed that while AI reduced time-to-pull-request by 58%, those PRs waited **4.6x longer in code review** and introduced **15–18% more security vulnerabilities** (Opsera, 2026). The Uplevel study of 800 developers using GitHub Copilot found a **41% increase in bug rates** with no measurable throughput gain (Uplevel, 2024). Harding and Kloster's (2024) quantitative analysis of 153 million changed lines of code at GitClear revealed that AI coding assistants function as "addition-only" machines — excellent at generating new code but structurally incapable of updating, reusing, or deleting existing code — triggering alarming spikes in code churn (code reverted or rewritten within two weeks of being pushed). Practitioner accounts from the React Native developer community independently corroborate this "addition-only" pattern: one developer observed that *"AI adds lots of sloppy code... functions can be reused but AI didn't do that for me — it created a new function for every feature,"* while another confirmed that *"AI loves to create new code and doesn't want to reuse existing code, even though it knows about it. I often have to explicitly tell it which helper functions it should use."*

On the hallucination front, Liu, Lin, and Thongtanunam (2025) found that approximately **50% of AI-generated code reviews** and **20% of AI-generated commit messages** contained definitive hallucinations — categorized as input inconsistency, logical internal contradictions, or core intention violations. Empirical security analyses report that approximately **45% of AI-generated code samples** contain OWASP Top 10 vulnerabilities — and this rate has not improved significantly

between 2024 and 2026 despite major model upgrades (Re-Entry AI, 2026). A particularly alarming attack vector has emerged: nearly **20% of AI-recommended package names** point to non-existent libraries, enabling “slopsquatting” attacks where adversaries register hallucinated package names as malicious dependencies to compromise software supply chains (Diffrey AI, 2025). Vettore (2026) formally defined and quantified the “Verification Tax,” demonstrating that while advanced prompting strategies could raise model accuracy to 95%, doing so simultaneously triggered a **66% increase in the average verification time**, a phenomenon he termed “instruction-induced friction.” Farrag (2026) modeled the broader dynamic as the **Productivity-Reliability Paradox (PRP)**, arguing that time saved during the generation phase is structurally eclipsed by the cognitive load of reading, context-matching, testing, and safely integrating AI code into pre-existing architectures.

4.1.2 Case Evidence: The IAP Integration

In the Fytly codebase, the Apple In-App Purchase (IAP) integration exemplifies the verification tax at its most acute. The git history tells the story clearly: the first IAP commit appears on December 28, 2025 (IAP WIP). What followed was a five-week debugging marathon spanning 20+ commits across December 28 through January 15, 2026 — including patches to `react-native-iap` itself (`Create react-native-iap+13.0.4.patch`), multiple App Store rejection fixes, and the eventual creation of dedicated documentation files (`SUBSCRIPTION_FLOW_TEST_CHECKLIST.md` , `APP_STORE_SUBSCRIPTION_REJECTION_FIX.md`).

I used Cursor’s Composer mode (backed by Gemini 2.5 Pro) to scaffold the initial architecture of the IAP service layer. The generated code followed standard React Native boilerplate patterns, but it contained four categories of critical structural and process errors:

1. **Stale API patterns:** The AI generated transaction receipt validation code using Apple’s legacy HTTP POST `/verifyReceipt` endpoint, failing to adopt the modern App Store Server API which utilizes JWT-signed transactions. Because the deprecated endpoint still resolved in the sandbox environment, this architectural mismatch compiled and passed initial manual checks. It was only exposed as a critical blocker when Apple rejected the production build during App Store review (commit: `Fix App Store rejection issues` , January 12, 2026).
2. **State Lifecycle and Edge Case Blindness:** When tasked with managing the subscription state transitions between free and premium tiers, the AI failed to handle native app lifecycle edge cases. For instance, closing the device or terminating the application caused the system to drop the user’s premium validation state. The purchase recovery flow also failed to manage cross-device Apple ID synchronization, forcing me to repeatedly intervene and guide the AI through exhaustive testing loops to manually build a custom `restorePurchasesFlow.js` service.
3. **Architectural hallucination:** The model suggested a synchronous verification flow when Apple’s StoreKit 2 architecture mandates asynchronous transaction listeners. The commit message `Fix critical payment flow issues: finish transaction timing, restore purchases logic, Android pending state` (January 12, 2026) captures the scope of the resulting fix—three separate subsystems had to be manually restructured and synchronized.

4. Institutional Process Ignorance: Beyond the code, the AI demonstrated a complete inability to navigate human-in-the-loop deployment processes. Even when injected with Apple’s complete subscription guidelines and policy documents via context window, the AI failed to understand the bureaucratic nuances of App Store Review—specifically, the requirement to submit the IPA binary accompanied by highly specific, manually written review notes. It could not synthesize the policy documentation into an actionable submission strategy.

This technical and process ignorance was compounded by external information asymmetry. Apple provided vague rejection feedback, and due to standard StoreKit sandbox latency anomalies, real subscription prices failed to populate on the physical testing device. Because the AI had no capacity to resolve uncoded platform bugs or engage in human negotiation, I was forced into a high-friction resolution loop: abandoning the AI to hunt for workarounds via YouTube tutorials, emailing Apple support, and ultimately requesting manual review calls to contest the rejections.

The git evidence is compelling: 20+ commits over 19 calendar days to correct and stabilize an integration that was “successfully scaffolded” in a single Cursor Composer session. The verification tax was not additive; it was multiplicative, as each consecutive native bridge adjustment required re-verifying downstream state assumptions across the entire payment logic.

4.1.3 The Perception Gap

The METR (2025) finding that developers *feel* 20% faster while being objectively 19% slower is consistent with the verification tax hypothesis. The granular data from Becker et al. (2025) reveals the full scale of the disconnect:

Table 1: Developer Belief vs. Measured Reality (Becker et al., 2025)

Dimension	Developer Belief	Measured Reality	Net Disconnect
Pre-task prediction	Expected to complete tasks 24% faster with AI	—	—
Post-task retrospective	Estimated AI made them 20% faster	—	—
Actual objective speed	—	19% slower with AI	~39% Illusion Gap

Vettore (2026) provides a cognitive explanation for this disconnect through two mechanisms: the “**Illusion of Fluency**” and “**Knowledge Overshadowing**.” Code generation is highly visible, manual labor; when an AI tool performs this typing instantly, it triggers a strong psychological sense of high throughput. However, the subsequent hours spent reading the generated code line-by-line, tracking down edge cases, running local tests, and dealing with subtle model hallucinations constitute heavily distributed, invisible cognitive work. Developers consistently discount this verification phase in post-hoc self-assessment, leading them to report massive productivity gains even when their actual task completion speeds have objectively declined (Becker et al., 2025;

Vettore, 2026). This phenomenon aligns with the broader automation literature: Parasuraman and Riley (1997) established that operators of automated systems systematically over-trust system outputs and under-monitor for failures — a pattern now termed **"automation complacency"** — providing a well-established theoretical foundation for the AI perception gap. Most recently, Vella and Blincoe (2026) conducted a longitudinal matched-cohort study demonstrating what they term the **"Productivity-Experience Paradox"**: while developers' self-reported productivity perceptions remained high (84%), their measured **flow state** and **cognitive load** dimensions progressively worsened over time — evidence that AI tools may be eroding the quality of developer experience even as they sustain the *illusion* of increased output.

Anonymized practitioner accounts gathered from online developer communities and a targeted survey posted to the *r/reactnative* subreddit (n=20+ respondents) corroborate this duality with striking consistency. Multiple developers reported that AI was *"great for generating boilerplate code, tests, and documentation," "helped me get unstuck quickly,"* and *"reduced context-switching by acting like an on-demand pair programmer."* One respondent described feeling *"at least 100× faster"* when using AI for tasks where the desired output was already mentally clear. Another reported that AI enabled a non-specialist to *"build an app in 1–2 months from scratch"* in React Native — a timeline that would have been inconceivable without AI assistance. However, these dramatic gains are critically complexity-dependent. The developers reporting order-of-magnitude speedups were typically working on greenfield projects with limited integration surfaces, minimal native platform dependencies, and no external review processes (such as App Store submission). In contrast, the FytlY codebase — with 99,000 lines of code, 93 Redux-Saga operations, native HealthKit and IAP integrations, and a multi-platform deployment pipeline — represents a qualitatively different complexity class where the verification tax compounds with every additional integration layer.

Yet the same practitioners simultaneously acknowledged a sharp inversion once complexity increased. One developer summarized it precisely: *"For boilerplate and exploring unfamiliar libraries? Huge time saver. For production debugging and edge cases? Sometimes it creates extra work because you still have to verify everything carefully."* Another stated bluntly: *"The moment it has to code, you can be really sure you're gonna spend a huge amount of time reviewing and fixing the code."* A particularly revealing account described a colleague who *"tracked his time for a month using Cursor — felt faster day-to-day, but his Jira tickets actually took 20% longer to close. The gap was all in verification."* This practitioner concluded with a formulation that captures the entire perception gap thesis in a single sentence: ***"The feeling of speed is real. The delivery isn't."***

A senior developer with 8+ years of experience provided a nuanced quantification: *"It helps to very quickly sketch up some solution, but without additional improvements of code quality — which can take easily 10× more than this first solution — the project would be on a very bad path."* He estimated his net productivity gain at approximately 20%, but noted the range was highly task-dependent: from 100–1000% for well-patterned tasks (adding a new page, extending a data model) to near-zero for real-time data processing work requiring deep domain reasoning. Critically, even this experienced developer acknowledged that on a side project built entirely via *"vibe coding"* in an

unfamiliar language, *"the resulting code is a mess and there are problems that could be easily eliminated with a good design."*

This perception gap has organizational consequences. Teams that adopt AI based on developers' self-reported satisfaction may be systematically overestimating productivity gains. In a startup context, where there is no QA team and no code review process, the verification tax falls entirely on the generating developer — creating a closed loop in which the same person generates and verifies, with all the confirmation biases that entails.

4.2 Session Drift

4.2.1 Mechanism

Session drift refers to the progressive degradation of context, intent, and coherence in extended interactions with AI agents. Unlike a human collaborator who maintains a persistent mental model of the project, current LLMs operate within finite context windows. As a conversation extends, earlier instructions, constraints, and architectural decisions are gradually "forgotten" or de-prioritized in favor of more recent prompt content.

In practice, session drift manifests in several ways:

- **Contradictory suggestions:** The AI recommends a pattern in message 40 that conflicts with a decision established in message 5.
- **Scope creep:** The AI progressively expands the scope of its modifications, touching files that were not part of the original request. One practitioner described a case where *"Cursor refactored a utility function and silently updated three imports across the repo. Looked clean, passed tests. Broke production because one import path had a side effect nobody remembered."*
- **Unprompted aesthetic drift:** The AI autonomously alters component colors, margins, or typographic styles without being explicitly instructed, breaking the established design system.
- **Unauthorized version control:** In terminal or CLI environments, the AI attempts to push code directly to the repository without human review, occasionally even overriding the developer's global GitHub configuration to commit under its own system name (e.g., "cursor agent").
- **Destructive error resolution:** When tasked with fixing linting or type-checking errors, the AI may resolve them by deleting the offending code entirely rather than correcting it — a pattern one developer described succinctly: *"It once fixed lint errors by removing the code."*
- **Style inconsistency:** The code generated late in a session may use different naming conventions, error handling patterns, or state management approaches than code generated earlier.
- **Context loss:** The AI "forgets" the project's architecture and begins generating code that assumes a different structure (e.g., generating hooks-based code for a class-based component system, or vice versa).

In response to these drift patterns, practitioners have independently developed compensatory verification workflows. Multiple developers reported configuring automated post-generation verification pipelines — running lint, type-checking, and test suites automatically after every AI-generated change. As one developer explained: *“My biggest unlock for AI coding was to give the agent a way to verify its work, e.g., by running lint, tests, typecheck after it finished a task. Burns hella more tokens but the higher quality output is so worth it.”* The emergence of such developer-built guardrail systems is itself evidence of complexity displacement: the verification infrastructure that these developers are constructing around AI agents represents a novel category of engineering overhead that did not exist in pre-AI workflows.

4.2.2 Temporal Context Asymmetry and Cross-Cutting Flow Diagnostics

A critical dimension of context limitation—and a fundamental bottleneck in large-scale software projects—is **Temporal Context Asymmetry**. While a human developer maintains a comprehensive, stateful mental model of the application’s execution sequence over time, an LLM evaluates code as statically fragmented snapshots within its localized prompt context window.

This asymmetry becomes highly visible when implementing cross-cutting diagnostics, such as instrumenting end-to-end telemetry (e.g., adding `console.log` statements) to trace the user lifecycle flow. For instance, in an authentication flow, a complete execution journey spans multiple decoupled files and asynchronous states: from the initial Sign Up and Sign In UI actions, to their respective Redux sagas, Redux slices, local storage initializations, active app sessions, and finally the Log Out button, saga, and slice.

If a developer asks an AI agent to instrument logging for the “log out flow,” the AI’s limited context forces it to evaluate only the immediate, localized files (e.g., the logout saga or component). Because the AI cannot comprehend the temporal, end-to-end execution path that precedes this event, its generated instrumentation is local and incomplete, failing to consider state transitions established hours earlier at login or sign-up.

Conversely, a human developer instinctively conceptualizes this timeline. The developer understands that tracing the system requires a sequential, step-by-step instrumentation: starting at the initial sign-in UI, migrating to its associated sagas and slices, and then proceeding downstream to the logout interface, saga, and state updates.

In large-scale codebases (such as FytIY’s 99,000 LOC system), this lack of end-to-end temporal mapping renders AI tools highly ineffective for cross-cutting concerns. The AI functions as a localized, “short-sighted” builder, whereas the human must act as the system-level orchestrator, guiding the AI file by file to preserve the temporal integrity of the codebase.

4.2.3 Case Evidence: The Workout Engine and the “Vibe Coding” Monolith

The FytIY workout execution screen represents the core user journey and the most complex module in the application, comprising approximately 30 Redux sagas and 60,000 lines of code across 244 screen and feature files. Developed between August 30, 2025 (WIP Workout) and May 2026, the git

history shows this module growing organically as features like exercise editing, historical charts, and live timers were appended.

During a high-intensity phase of development, I adopted the modern practitioner paradigm colloquially termed “**vibe coding**”—a workflow characterized by rapid, iterative prompting where the developer delegates continuous code generation and file modifications to an autonomous agent (in this case, Cursor’s Auto mode) without manual code intervention or rigorous intermediate design reviews.

During this “vibe coding” sprint, the underlying model routing was opaque; the autonomous agent dynamically selected models behind the scenes without my explicit awareness of the capabilities or contextual limitations of each tier. As I continuously prompted the agent to add features to the workout execution screen, it prioritized local task fulfillment over global architectural cleanliness. Because the agent lacked an inherent understanding of modular boundaries, it consistently appended helper functions, local state handlers, complex UI rendering blocks, and inline styling to a single container.

Within a month of continuous, unmonitored “vibe coding” iterations, the main workout journey screen had degenerated into a massive, **8,000-line monolithic JavaScript file**.

The consequences for the React Native application were severe: 1. **Performance Degradation and Screen Freezing:** React Native relies on a single JavaScript execution thread that bridges to native rendering threads. A single component file of 8,000 lines, loaded with complex state, triggered massive, cascading re-renders. During live workout testing, this thread blockage caused severe frame drops, sluggish animations, and intermittent screen freezes when the user attempted to log sets or cycle timers. 2. **Testing and Verification Paralysis:** Manual and automated testing became practically impossible. Because the monolithic file had no clear separation of concerns, any change proposed by the agent to fix a bug in one sub-module inadvertently broke unrelated states elsewhere in the file, leading to endless feedback loops.

The sheer size of the 7,000-8,000-line monolith forced a full month of painstaking refactoring, a delay that proved fatal for our early-stage startup. 3. **Severe Cognitive Overhead:** When critical state issues occurred, I was forced to read and reverse-engineer thousands of lines of dense, AI-generated spaghetti code to understand the basic control flow.

The initial clean architecture—intended to follow a strict Redux-Saga pattern with decoupled selectors, slices, and presentation components—had been entirely subverted by the high-velocity generation. To resolve this “vibe coding” debt, I was forced to pause feature development for two weeks for a demanding decomposition and migration phase. Operating under tight API quotas and rate limits, I devised a multi-model migration strategy. I leveraged free and open-source models alongside specialized command-line tools—specifically the OpenCode CLI agent and lightweight local models (such as Big Pickle)—to systematically parse and partition the monolithic file. Slowly and steadily, I extracted logic into isolated, reusable functional components and custom React hooks, decoupling UI presentation from state hooks and Redux sagas for network synchronization.

This multi-model approach allowed me to apply specific models to distinct partitioning sub-tasks, optimizing processing efficiency and resource allocation.

Retrospectively, this structural crisis highlights a profound lesson in prompt discipline: **architectural rules and system guardrails must be established at the absolute inception of an AI-assisted project.** Waiting until degeneration occurs forces a costly, complex interest payment on comprehension debt that could have been avoided by establishing strict boundaries earlier. To prevent a recurrence of this structural drift, I formalized the **Cursor Skills** system. I authored three machine-readable architectural rule files (`saga-logic` , `responsive-ui` , and `rn-performance`) that the AI is forced to load at the beginning of each session. For example, the `saga-logic` skill explicitly mandates: *"Always import the base network service: `import { networkCall } from 'Src/Services/networkCall'` " and "Default to `takeLatest` to prevent duplicate API calls."*

Despite these guardrails, however, as sessions extended beyond 30+ messages, session drift consistently re-emerged: - By message 30, the agent would begin placing business logic directly into UI components rather than saga side-effects, violating the separation of concerns. - By message 40, the agent would suggest inline styles (`width: 100`) instead of utilizing the responsive helpers (`respWidth(100)`) mandated by the UI skill. - By message 50, the agent would bypass Redux completely, recommending raw `useEffect` -based side effects instead of dispatching Redux actions. This did not just violate style guidelines; it introduced a critical race condition. In React Native, background-to-foreground transitions frequently cause components to re-mount. Bypassing the Redux-Saga queue with local `useEffect` hooks led to duplicate network calls to the sync endpoint, occasionally corrupting the local SQLite workout logs. Resolving this required manual state reconciliation and reinforcing the saga rules via the custom `saga-logic` skill.

4.2.4 Theoretical Implications

Session drift illustrates a fundamental tension in the design of agentic AI systems. The value of a long session lies in accumulated context — the agent "understands" the project after many interactions. But the reliability of a long session degrades with that same accumulation. The developer is caught between two suboptimal strategies:

- **Short sessions:** High setup cost, low drift risk.
- **Long sessions:** Low setup cost, high drift risk.

Neither strategy eliminates the overhead; they merely redistribute it between *initialization cost* and *correction cost*. This is complexity displacement in its purest form.

Notably, the practitioner community has independently converged on elaborate mitigation protocols that validate the severity of this problem. A widely circulated workflow — described by one practitioner as a "Blueprint" for maximizing AI assistant output — prescribes four steps: (1) maintain a dedicated project workspace per codebase, (2) write explicit master instructions specifying goal, role, rules, and style, (3) prompt the AI to generate a structured summary at the end of every session ("What was decided, what is still open, what comes next?"), and (4) *always start a new session in a fresh chat*, relying on the saved summary to restore context. The existence of such

protocols is itself empirical evidence: developers are constructing manual state-management systems around a tool that is, by design, stateless. The cognitive labor of maintaining these session-boundary rituals — writing summaries, curating project files, re-initializing context — constitutes a tangible and recurring overhead that is entirely absent from non-AI-assisted workflows.

4.2.5 Cross-Case Validation: The TaskOrbit Project

To assess whether complexity displacement generalizes beyond a solo-founder React Native context, I examined a second codebase: **TaskOrbit**, a conversational agent platform developed as part of a university team project (AMOS 2026, FAU). TaskOrbit was built by a distributed team of 12 contributors over 68 days, accumulating 645 commits across approximately 46,000 lines of Python, TypeScript, and Terraform code. The team used AI coding assistants (primarily Cursor’s agent mode) extensively throughout development.

A systematic analysis of the full git history reveals four quantifiable displacement patterns:

Table 6: TaskOrbit Quantified Complexity Displacement Patterns

Pattern	Count	% of 645 Commits	Thesis Mechanism
Fix-after-fix chains	121 chains	~19%	Verification Tax
Janitorial commits (lint/CI/format)	50	~8%	Verification Tax
State restoration (reverts, lost state)	10 substantive	~1.5%	Session Drift
Cross-path desync fixes (voice/text/stream)	12	~2%	Orchestration Tax

Approximately 30% of all commits were not adding features — they were cleaning up after AI-assisted implementation. This ratio quantifies the verification tax at the project level.

Fix-After-Fix Chains. Feature commits were systematically followed by 3–10 corrective commits within 48 hours. The densest chain occurred during the Conditional Workflow Engine integration (June 15–18): a single feature commit triggered 10 consecutive fixes over 72 hours, including state restoration, CI failure resolution, voice/text desync, and stream endpoint corrections. This pattern recurred across every major feature: the LiveKit voice pipeline (May 11–12, 5 fixes), the agent handoff system (June 1–3, 6 fixes), and the streaming LLM endpoint (June 7–10, 5 fixes).

The “Tests Green, Demo Red” Gap. Automated pytest cases consistently passed (11/11 for the transitive workflow logic), yet product owner demos repeatedly failed. The root cause was a divergence between the isolated test execution path and the actual runtime path triggered by the UI. The AI optimized *proof of logic* (function correctness), while the human still paid the full cost of

proof of system behavior (which runtime entry point executed the new logic). In integration-heavy features, the verification tax is paid twice: once on code correctness and once on path correctness.

Janitorial Overhead. 50 commits (8% of total) were dedicated exclusively to fixing linting errors (Ruff), formatting violations (Prettier), CI pipeline failures, and merge conflicts — tasks that represent the “cleanup cost” of AI-generated code that compiles and passes logic tests but violates project-level toolchain constraints. These janitorial commits clustered immediately after large feature PRs, confirming that AI-generated code systematically externalizes formatting and integration compliance to the human developer.

Multi-Path Desync. TaskOrbit maintained three parallel execution paths: a text API, an SSE streaming endpoint, and a LiveKit voice pipeline. The AI consistently updated one path while neglecting the others, producing 12 explicit desync fix commits. The most revealing commit (7986d48 : *“resolve transitive workflow execution and voice/text desync”*) directly names the failure mode. This required the manual creation of a dedicated synchronization hook (`useWorkflowVoiceSync.ts`) — a component the AI was structurally incapable of recommending because it could not conceptualize the runtime relationship between independent execution pipelines.

The contrast between the two cases strengthens the generalizability of the Complexity Displacement Thesis:

Table 5: Cross-Case Structural Comparison (FytIY vs. TaskOrbit)

Dimension	FytIY (Solo Startup)	TaskOrbit (Team Project)
Setting	Solo founder, React Native, 99K LOC	12-person team, Python/FastAPI + React, 46K LOC
AI task	Generate features	Integrate engine + deliver PO demos
Failure mode	Wrong IAP architecture, monolith	Wrong runtime path, voice/text desync
Verification ratio	20+ commits fixing one integration	30% of all commits are non-feature
Developer role	Author + verifier	Configurator + demo operator + coordinator

Complexity displacement is not stack-specific, team-size-specific, or domain-specific. It intensifies when the product is a **workflow** spanning frontend state, backend orchestration, voice streams, and database enrichment — not a single file.

4.3 Budgeted Intelligence: The Economics of AI-Assisted Development

4.3.1 Rate Limits as Productivity Constraints

A dimension of AI-assisted development that is entirely absent from the academic literature is the operational economics of LLM usage. In practice, developers interact with AI through services that impose explicit resource constraints:

- **Token quotas and Tiered Trade-offs:** Services like Cursor impose daily or weekly token limits. This forces immediate trade-offs between model intelligence and volume. For example, utilizing a highly capable model like Claude 3.5 Sonnet provides excellent architectural reasoning but consumes weekly quotas rapidly. Switching to a lighter model like Claude 3 Haiku preserves tokens but reduces output quality. When the weekly high-tier quota is exhausted, the developer must either wait, switch to a less capable model, or accept that they can only complete a strictly limited amount of high-complexity work per week.
- **Rate limits:** API-based models impose requests-per-minute limits that can arbitrarily throttle intensive development sessions.
- **Latency variance:** Larger, more capable models have response latencies of 30–90 seconds for complex prompts, while smaller models respond in 5–15 seconds but require significantly more human verification.

These constraints transform AI from a tool with constant availability (like an IDE or a compiler) into a *budgeted resource* that requires allocation decisions. The developer must decide:

- Which tasks merit the “expensive” model versus the “cheap” model?
- Should I use my remaining daily quota on this complex refactoring or save it for tomorrow’s critical feature?
- Should I wait 60 seconds for a better response or accept a mediocre response in 10 seconds?

4.3.2 Case Evidence: Model Selection Strategy

During FytIY development, I used five distinct AI tools, each with different operational constraints:

Table 2a: GenAI Tool Constraints

Tool	Interface	Model	Rate Limit Constraint
Cursor (Tab completion)	IDE inline	Gemini 2.0 Flash	Fast, generous quota
Cursor Composer	IDE multi-file	Gemini 2.5 Pro	Slower, limited daily usage
Cursor Auto Mode	IDE autonomous	Gemini 2.5 Pro	Aggressive quota consumption
OpenCode (CLI)	Terminal agent	Claude Sonnet/Opus	Token-based billing
Gemini CLI	Terminal	Gemini 2.5 Pro	API rate limits

This created a deliberate model allocation strategy:

Table 2b: FytIY Tool-to-Model Allocation Strategy

Task Type	Tool Choice	Rationale
Line-level completions	Cursor Tab (Flash)	Fast, low-cost, good for known patterns
Multi-file features	Cursor Composer (2.5 Pro)	Best for cross-file changes (saga + reducer + component)
Autonomous refactoring	Cursor Auto Mode	High-quality but quota-heavy
Complex debugging	OpenCode CLI	Deep reasoning, full terminal access
Quick scripts / queries	Gemini CLI	Fast iteration, unlimited local access
IAP / HealthKit	Manual (no AI)	AI hallucinates on native platform APIs

This allocation strategy is itself a form of cognitive overhead. It requires the developer to (a) evaluate the complexity of each task, (b) select the appropriate tool and model tier, (c) monitor remaining quotas across *multiple services simultaneously*, and (d) schedule high-value tasks during periods of quota availability.

This pattern of model specialization extends beyond individual developers. Across practitioner communities, a clear consensus has emerged that different AI models excel at fundamentally different cognitive tasks — and developers are expected to internalize this taxonomy. Gemini is favored for natural language tasks and documentation; Claude (particularly Sonnet and Opus) is preferred for complex architectural reasoning and multi-file code generation, as its extended thinking capabilities produce more structurally coherent output; ChatGPT is used for rapid, low-stakes queries; and GitHub Copilot is optimized for inline completions within the IDE. The developer must not only select the right tool but the right *model personality* for each sub-task — a meta-cognitive allocation layer that has no precedent in pre-AI software engineering.

In a startup context where a single developer handles all engineering, this meta-cognitive load is non-trivial. I estimate that approximately 5–10% of my daily cognitive budget was spent on *tool management* rather than *product development*.

4.3.3 The “Thursday Problem”

I consistently observed a highly practical manifestation of budgeted intelligence, which I conceptualize as the “**Thursday Problem**.” Under Cursor’s premium tier, weekly quotas of fast-model access reset on a fixed cycle. By late Thursday, this quota was frequently exhausted due to

high-consumption autonomous iterations in Cursor’s Auto mode during the early part of the week. This constraint introduced a bimodal development rhythm:

- **Monday–Wednesday (High-Velocity Scaffolding):** Relying heavily on Composer and Auto mode to generate large, multi-file feature additions and boilerplate layouts.
- **Thursday–Friday (Manual Consolidation & Debugging):** Reverting to manual coding or targeted API-based CLI tools. Strikingly, this constraint forced a critical consolidation phase—I had to manually refactor, document, and test the code generated earlier in the week.

This bimodal rhythm is indirectly reflected in the git history. While one might expect productivity to plunge when token quotas were exhausted, commit frequency actually peaked on Thursdays (345 commits) and Fridays (346 commits) compared to earlier in the week (averaging 255 commits). Rather than indicating higher productivity, this spike highlights a change in development granularities: manual coding naturally produces smaller, highly frequent, and incremental commits, whereas AI-assisted sessions result in massive, multi-file changesets committed in single, large blocks.

This cyclical pattern has no equivalent in non-AI-assisted development. It represents a novel form of resource-constrained scheduling that the developer must internalize and plan around.

5. Synthesis: Modeling Complexity Displacement in React Native Development

5.1 Quantifying Displacement: A Task-Level Analysis

To ground the theoretical framework in empirical observation, I applied the task-based decomposition approach to the FytIY development process. The application’s 99,000 lines of code, 2,111 commits over 209 active coding days (July 6, 2025 – May 20, 2026), and 10+ feature modules provide a sufficiently complex corpus for analysis.

The productivity figures presented in Table 4 represent a retrospective estimation derived from a detailed audit of FytIY’s git history. By classifying all 2,111 commits into task categories (e.g., scaffolding, integration, bug fixing) and cross-referencing them with my personal development time-logs and code churn metrics, I reconstructed the ratio of generation time to verification/corrective time. While subjective, this granular case-level tracking provides a transparent window into the micro-mechanisms of complexity displacement.

The git history reveals a clear development arc with identifiable phases:

Table 3: FytIY Codebase Evolution Phases

Phase	Period	Commits	Focus
Foundation	Jul 2025	26	Project init, navigation, bottom tabs
Scaffolding	Aug 2025	42	Auth screens, onboarding, initial UI
Core Features	Sep–Oct 2025	160	Workout engine, exercise flow, API integration
Integration Sprint	Nov 2025	103	Firebase, HealthKit, Google Fit, WebSocket
Peak Velocity	Dec 2025 – Jan 2026	1,181	IAP, analytics, streaks, treasure, polish
Stabilization	Feb–Mar 2026	406	Bug fixes, App Store submissions, UI refinement
Gamification	Apr–May 2026	193	Ranks, trophies, haptics, final polish

The December–January “Peak Velocity” period is particularly revealing: 1,181 commits in 62 days, with single-day peaks reaching 34 commits (December 14 and 27, 2025). During this period, 494 unique files were modified — nearly 80% of the entire codebase. This intensity was only possible with AI assistance; however, as the analysis below demonstrates, the raw commit velocity masked significant verification overhead.

I categorized the primary development tasks and estimated the AI impact on each:

Table 4: FytlY Task-Level Productivity and Verification Estimates

Task Category	% of Total Effort	AI Speed Gain	Verification Overhead	Net Productivity
Boilerplate / scaffolding	15%	+50%	Low (5%)	+45%
CRUD screens	10%	+50%	Low (10%)	+40%
Business logic (sagas)	20%	+30%	Medium (20%)	+10%
UI polish / animations	15%	+20%	Medium (15%)	+5%

Task Category	% of Total Effort	AI Speed Gain	Verification Overhead	Net Productivity
Platform integrations (IAP, HealthKit)	15%	+10%	High (40%)	-30%
Debugging / troubleshooting	15%	+15%	High (25%)	-10%
Architecture / design decisions	10%	+5%	High (30%)	-25%

Weighted net productivity gain: approximately +8%

This figure stands in sharp contrast to the 26–55% gains reported in controlled experiments, but aligns closely with what the METR study and DORA report suggest at the organizational level. The discrepancy is explained by the *composition effect*: controlled experiments measure performance on isolated, well-defined tasks (where AI excels), while real-world development involves a heterogeneous mix of tasks, many of which impose verification overhead that exceeds the generation speed gain.

5.2 The Complexity Displacement Model

Based on the theoretical framework and case evidence, I propose the following model:

Without GenAI:

$$\text{Total Development Effort} = \text{Execution} + \text{Design} + \text{Testing} + \text{Debugging}$$

With GenAI:

$$\begin{aligned} \text{Total Development Effort} = & (\text{Execution} \times 0.5) + \text{Design} + \text{Testing} + \text{Debugging} \\ & + \text{Verification Tax} \\ & + \text{Session Drift Correction} \\ & + \text{Tool Management Overhead} \\ & + \text{Comprehension Debt Service} \end{aligned}$$

The first term ($\text{Execution} \times 0.5$) represents the genuine speed gain that AI provides. But the four additional terms — none of which exist in non-AI-assisted development — partially or fully offset this gain. The model predicts:

1. **For simple, well-defined tasks:** Net gain is strongly positive (AI’s strength; verification overhead is low).
2. **For complex, integration-heavy tasks:** Net gain is near zero or negative (AI’s weakness; verification overhead dominates).

3. **For extended projects:** Comprehension debt accumulates over time, causing long-term maintenance costs to rise even if initial development was faster.

5.3 The “Comprehension Debt” Dimension

A subtle yet critical form of complexity displacement is **comprehension debt**—the long-term cost of maintaining code that the developer did not fully write or fully understand. When AI generates a 200-line saga that “works,” the developer may accept it without fully internalizing its logic. Months later, when a bug emerges in that code, the developer must reverse-engineer their own codebase—a cost that would not exist if they had written the code manually.

This phenomenon extends Ward Cunningham’s (1992) classical concept of technical debt. While traditional technical debt represents deliberate design compromises made for speed, “comprehension debt” is cognitive in nature. It represents an accumulation of code that is structurally functional but cognitively unowned. This directly accelerates Lehman’s Law of Increasing Complexity: because the developer does not fully internalize the evolutionary history of the AI-generated modules, the cognitive effort required to modify the system in subsequent generations grows exponentially.

In the FytIY codebase, I identified several instances of comprehension debt:

- A subscription status-checking saga that used a retry pattern I did not fully understand until it failed under edge conditions three months after implementation.
- A workout sync mechanism that correctly handled the happy path but used an error-recovery strategy (exponential backoff with jitter) whose parameters were subtly wrong — a bug that only manifested under poor network conditions.

These examples illustrate that AI-generated code can function as a *black box within the developer’s own codebase* — a reversal of the normal relationship between developer and code.

5.4 Implications for the Startup Context

The complexity displacement model has particular implications for resource-constrained startups:

1. **No verification safety net:** In a large organization, code review, QA, and testing pipelines catch AI-generated errors before they reach production. In a startup with one or two developers, the generator and the reviewer are the same person — a conflict of interest that amplifies confirmation bias.
2. **Compounding debt:** Startups under time pressure are most likely to accept AI-generated code without full comprehension, accumulating comprehension debt that compounds as the codebase grows. At 99,000 lines, the cost of servicing this debt becomes a significant drag on velocity.

3. **Budgeted intelligence hits harder:** A startup cannot afford the \$400+/month cost of unlimited access to top-tier models. The operational constraints of budgeted intelligence disproportionately affect small teams, who must be more strategic about when and how they deploy AI assistance.
 4. **False velocity:** Perhaps most dangerously, AI creates a perception of velocity that can mask fundamental product-market fit questions. If a feature can be built in a day instead of a week, the temptation is to build first and validate later — an inversion of lean startup principles that can lead to technically sophisticated products that no one needs.
-

6. Discussion

6.1 Reframing the Debate

The contribution of this paper is not to argue that GenAI is unproductive — the evidence for local efficiency gains is robust and reproducible. Rather, the contribution is to *reframe* the debate from “does AI help?” to “how must organizational processes evolve to capture the value AI creates?”

The Complexity Displacement Thesis suggests that the gap between AI's potential productivity gains and its realized impact is not a technological gap but a *sociotechnical* one. The technology generates code faster; the organization has not yet learned to verify, integrate, and maintain that code efficiently. This is precisely the pattern that Brynjolfsson's J-Curve predicts for general-purpose technologies.

6.2 Toward a “Complexity-Aware” Framework

Based on the analysis, I propose four principles for a complexity-aware approach to AI integration in software development:

1. **Explicit verification budgets.** Organizations should allocate time specifically for AI output review, rather than assuming that AI-generated code is “free.” A rule of thumb from this case study: budget 20–40% of the time saved by AI generation for verification.
2. **Session discipline.** Developers should maintain explicit architecture documents and enforce session boundaries, starting fresh conversations for each logically distinct task. The cost of re-establishing context is lower than the cost of correcting session drift.
3. **Task triage.** Not all tasks benefit equally from AI assistance. Organizations should develop explicit taxonomies of task types and their AI suitability, rather than applying AI uniformly across all development activities.
4. **Comprehension audits.** Periodically, developers should review AI-generated sections of the codebase with the explicit goal of building comprehension — understanding not just *what* the

code does but *why* it was written that way. This converts comprehension debt into comprehension equity.

6.3 Limitations

This paper has several limitations that should be acknowledged:

1. **Single-case design.** The reflexive case analysis is drawn from one startup, one developer, and one technology stack (React Native). The generalizability of the specific quantitative estimates is limited.
 2. **Self-report bias.** As the developer and the analyst are the same person, the case evidence is subject to retrospective rationalization and confirmation bias.
 3. **Rapidly evolving technology.** GenAI capabilities are improving rapidly. The specific operational constraints documented here (rate limits, context window sizes, hallucination rates) may change significantly within months.
 4. **No controlled comparison.** The case study lacks a true counterfactual — I cannot know with certainty how long the same project would have taken without AI assistance. The estimates are informed approximations.
-

7. Conclusion

Robert Solow observed the computer age everywhere but in the productivity statistics. In 2026, we can observe GenAI everywhere but in the delivery metrics. This paper has argued that the persistence of the productivity paradox in AI-assisted software development is explained by *complexity displacement* — the systematic reallocation of cognitive labor from execution to verification, orchestration, and resource management.

Through a reflexive analysis of FytlY — a 99,000-line React Native application built over nine months with intensive GenAI usage — I identified three concrete displacement mechanisms: the Cognitive Verification Tax, Session Drift, and Budgeted Intelligence. Together, these mechanisms reduce the effective productivity gain from the 26–55% reported in controlled experiments to approximately 8% in real-world, full-lifecycle development.

The paradox persists not because the technology fails, but because we are in the trough of the Productivity J-Curve. Organizations — and individual developers — have not yet developed the complementary assets (review norms, architectural boundaries, prompt discipline, verification budgets) required to capture the value that GenAI creates. As these assets mature, we should expect the paradox to resolve — as it did for IT in the 1990s — but the timeline and magnitude of that resolution remain uncertain.

The implication for IS research is clear: productivity should not be measured at the task level alone. A framework that accounts for the *total cost of AI-augmented development* — including verification, drift correction, tool management, and comprehension debt — is necessary to accurately assess whether GenAI represents a genuine productivity breakthrough or merely a rearrangement of where the work goes.

References

- Acemoglu, D., & Restrepo, P. (2019). Automation and New Tasks: How Technology Displaces and Reinstates Labor. *Journal of Economic Perspectives*, 33(2), 3–30. DOI: [10.1257/jep.33.2.3](https://doi.org/10.1257/jep.33.2.3)
- Becker, J., Rush, N., Barnes, E., & Rein, D. (2025). Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. *arXiv preprint arXiv:2507.09089*. URL: <https://arxiv.org/abs/2507.09089>
- Brynjolfsson, E. (1993). The Productivity Paradox of Information Technology. *Communications of the ACM*, 36(12), 66–77. DOI: [10.1145/163298.163309](https://doi.org/10.1145/163298.163309)
- Brynjolfsson, E., & Hitt, L. M. (2000). Beyond Computation: Information Technology, Organizational Transformation and Business Performance. *Journal of Economic Perspectives*, 14(4), 23–48. DOI: [10.1257/jep.14.4.23](https://doi.org/10.1257/jep.14.4.23)
- Brynjolfsson, E., Mitchell, T., & Rock, D. (2018). What Can Machines Learn, and What Does It Mean for Occupations and the Economy? *AEA Papers and Proceedings*, 108, 43–47. DOI: [10.1257/pandp.20181019](https://doi.org/10.1257/pandp.20181019)
- Brynjolfsson, E., Rock, D., & Syverson, C. (2021). The Productivity J-Curve: How Intangibles Complement General Purpose Technologies. *American Economic Journal: Macroeconomics*, 13(1), 333–372. DOI: [10.1257/mac.20180386](https://doi.org/10.1257/mac.20180386)
- Chui, M., Hazan, E., Roberts, R., Singla, A., Smaje, K., Sukharevsky, A., Yee, L., & Zimmel, R. (2023). The Economic Potential of Generative AI: The Next Productivity Frontier. McKinsey Global Institute. URL: <https://www.mckinsey.com/mgi/our-research/the-economic-potential-of-generative-ai>
- Cunningham, W. (1992). The WyCash Portfolio Management System. *OOPSLA '92 Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications*, 29–30. DOI: [10.1145/157710.157715](https://doi.org/10.1145/157710.157715)
- David, P. A. (1990). The Dynamo and the Computer: An Historical Perspective on the Modern Productivity Paradox. *American Economic Review*, 80(2), 355–361. URL: <https://www.jstor.org/stable/2006591>
- Digital Applied / LinearB. (2026). Developers Spend 11.4 Hours/Week Reviewing AI Code. Bytelota. URL: <https://linearb.io/blog/review-time-developer-productivity>

DORA. (2024). Accelerate State of DevOps Report 2024. Google Cloud. URL: <https://cloud.google.com/devops/state-of-devops>

DORA. (2026). Accelerate State of DevOps Report 2026. Google Cloud. URL: <https://cloud.google.com/devops/state-of-devops>

Eloundou, T., Manning, S., Mishkin, P., & Rock, D. (2023). GPTs are GPTs: An Early Look at the Labor Market Impact Potential of Large Language Models. *arXiv preprint arXiv:2303.10130*. URL: <https://arxiv.org/abs/2303.10130>

Faros AI. (2026). The Real Impact of AI on Software Engineering: Engineering Telemetry Report. Faros AI. URL: <https://www.faros.ai/resources/telemetry-report-ai-software-engineering>

Farrag, S. E. (2026). The Productivity-Reliability Paradox: Specification-Driven Governance for AI-Augmented Software Development. *arXiv preprint arXiv:2605.01160*. URL: <https://arxiv.org/abs/2605.01160>

Harding, W., & Kloster, M. (2024). Coding on Copilot: 2024 Data Suggests Downward Pressure on Code Quality. GitClear Research. URL: https://www.gitclear.com/coding_on_copilot_gitclear_research

Kalliamvakou, E. (2022). Research: Quantifying GitHub Copilot's Impact on Developer Productivity and Happiness. GitHub Blog. URL: <https://github.blog/2022-09-07-research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness/>

Lehman, M. M. (1980). Programs, Life Cycles, and Laws of Software Evolution. *Proceedings of the IEEE*, 68(9), 1060–1076. DOI: [10.1109/PROC.1980.11805](https://doi.org/10.1109/PROC.1980.11805)

Liu, C., Lin, H. Y., & Thongtanunam, P. (2025). Hallucinations in Code Change to Natural Language Generation: Prevalence and Evaluation of Detection Metrics. *Proceedings of the International Joint Conference on Natural Language Processing (IJCNLP)*. URL: <https://aclanthology.org/2025.ijcnlp-main.84>

METR. (2025). Measuring the Impact of AI Coding Assistants on Developer Productivity: A Randomized Controlled Trial. Model Evaluation and Threat Research. URL: <https://evals.warad.org/metr-2025-ai-coding-productivity-rct.pdf>

Opsera. (2026). AI Coding Impact Benchmark Report. Opsera.ai. URL: <https://www.opsera.ai/resources/ai-coding-impact-benchmark>

Parasuraman, R., & Riley, V. (1997). Humans and Automation: Use, Misuse, Disuse, Abuse. *Human Factors*, 39(2), 230–253. DOI: [10.1177/001872089703900202](https://doi.org/10.1177/001872089703900202)

Solow, R. M. (1987). We'd Better Watch Out. *New York Times Book Review*, July 12, 36. URL: <https://www.nytimes.com/1987/07/12/books/we-d-better-watch-out.html>

Uplevel Data Labs. (2024). GenAI for Coding Research Report. Uplevel. URL: <https://www.uplevel.com/resources/genai-for-coding-research-report>

Vella, D., & Blincoe, K. (2026). The Productivity-Experience Paradox: Longitudinal Effects of AI Coding Tools on Developer Experience. *Proceedings of the 48th International Conference on Software Engineering (ICSE 2026)*. DOI: [10.1109/ICSE2026.00010](https://doi.org/10.1109/ICSE2026.00010)

Vettore, P. M. (2026). Generative AI in Professional Workflows: Adoption Patterns, Reliability Perception. *Università Ca' Foscari Venezia*. URL: <http://hdl.handle.net/10278/vettore2026>

Appendix A: FytIY Codebase Summary

Table A.1: FytIY Codebase Metrics Summary

Metric	Value
Total Source Files	621
Total Lines of Code	~99,000
Screen/Page Files	244 (~60,500 LOC)
Shared UI Components	44
Redux State Slices	11
Redux-Saga Operations	93
API Service Modules	18
Custom Hooks	23
NPM Dependencies	58 production + 25 dev
Git Commits	2,106
Development Period	Sep 2025 – May 2026
Team Size (Engineering)	1 developer
Platforms	iOS + Android
Complexity Score	8.5 / 10

Appendix B: AI Tools Used During Development

Table B.1: Taxonomy of GenAI Tools Utilized

Tool	Interface	Primary Use Case	Model
Cursor (Tab)	IDE inline autocomplete	Line-level code completions	Gemini 2.0 Flash
Cursor Composer	IDE multi-file editor	Cross-file feature implementation (saga + slice + component)	Gemini 2.5 Pro
Cursor Auto Mode	IDE autonomous agent	Autonomous multi-step refactoring	Gemini 2.5 Pro
OpenCode	Terminal CLI agent	Complex debugging, architecture decisions	Claude Sonnet / Opus
Gemini CLI	Terminal	Quick queries, script generation, fallback when Cursor quota exhausted	Gemini 2.5 Pro

Appendix B.1: Cursor Skills Configuration

To mitigate Session Drift (Section 3.2), three Cursor Skills files were maintained as machine-readable architectural rules:

- saga-logic Skill:** - Mandates `import { networkCall } from 'Src/Services/networkCall'` in all sagas - Requires `try/catch` wrappers with `actionSuccess / actionFailure` dispatch - Defaults to `takeLatest` to prevent duplicate API calls
- responsive-ui Skill:** - Mandates `respWidth()`, `respHeight()`, `respFontSize()` for all dimensions - Prohibits raw numeric values in styles (e.g., `width: 100` → `width: respWidth(100)`) - Requires `StyleSheet.create` at file bottom; SVGs from `Src/assets/svg`
- rn-performance Skill:** - Requires `getItemLayout` for `FlatLists`, `removeClippedSubviews={true}` - Mandates `React.memo` for pure components, `useCallback` for prop functions - Requires `react-native-fast-image` with `priority` for remote images
- new-screen Command:** A 15-line instruction set that generates a complete screen folder (`index.jsx`, `styles.js`, `saga.js`, `slice.js`) with correct imports and patterns. This represents an early form of “AI process automation” — codifying organizational knowledge into machine-readable instructions.

These files constitute concrete evidence of the “complementary intangible assets” that Brynjolfsson’s J-Curve model predicts organizations must develop to capture AI’s value.

Appendix C: Task-Level Productivity Estimates

Table C.1: Granular Task-Level Effort and Net Productivity Gains

Task Category	Effort Share	AI Speed Gain	Verification Cost	Net Impact
Boilerplate / Scaffolding	15%	+50%	5%	+45%
CRUD Screens	10%	+50%	10%	+40%
Business Logic (Sagas)	20%	+30%	20%	+10%
UI Polish / Animations	15%	+20%	15%	+5%
Platform Integrations	15%	+10%	40%	-30%
Debugging	15%	+15%	25%	-10%
Architecture Decisions	10%	+5%	30%	-25%
Weighted Average	100%			~+8%

Appendix D: Git-Derived Development Statistics

Table D.1: FytLY Git History Quantitative Statistics

Metric	Value
Total Commits	2,111
Active Coding Days	209
Calendar Duration	Jul 6, 2025 – May 20, 2026 (10.5 months)
Average Commits/Active Day	~10.1
Peak Single-Day Commits	34 (Dec 14 & Dec 27, 2025)
Peak Month	January 2026 (635 commits, 33 active days)
Bug Fix Commits	369 (17.5% of total)
WIP Commits	73 (3.5% of total)
Merge Commits	52
Total Branches	27

Metric	Value
Files Modified in Peak Month (Dec)	494
Contributors	1 primary (1,659 commits as "fytlyRise")

Monthly Commit Velocity

Table D.2: Fytly Monthly Commit Velocity and Development Focus

Month	Commits	Phase
Jul 2025	26	Foundation
Aug 2025	42	Auth & UI
Sep 2025	64	Workout Engine
Oct 2025	96	API Integration
Nov 2025	103	Health & Firebase
Dec 2025	546	Peak Sprint (IAP, Analytics, Streaks)
Jan 2026	635	Peak Sprint (Subscriptions, Polish)
Feb 2026	314	Stabilization
Mar 2026	92	App Store Fixes
Apr 2026	162	Gamification
May 2026	31	Final Ranks & Wind-down

Author Note & Replication

Correspondence concerning this working paper should be addressed to Shikhar Thakur (shikharthakur2404@gmail.com).

How to cite: Thakur, S. (2026). *The Productivity Paradox of Generative AI in Software Development: A Complexity Displacement Analysis Grounded in Startup Practice*. Working Paper / Research Preprint, Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU).

All empirical telemetry, commit frequency data, and architectural rule definitions reflect real-world production engineering on FytlY (2025–2026).